

✓ Multiple Linear Regression: Multi-Channel Marketing Analysis

Objective: Analyse a marketing dataset with three advertising channels (TV, Radio, Social Media) and an Influencer variable to build a Multiple Linear Regression model. We will check for multicollinearity, select significant predictors, validate OLS assumptions, and deliver a prioritised, evidence-based budget recommendation.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12

print('Libraries loaded successfully.')
```

Libraries loaded successfully.

```
df_raw = pd.read_csv('marketing_and_sales_data.csv')
print(f'Dataset shape: {df_raw.shape}')
df_raw.head(10)
```

Dataset shape: (572, 5)

	TV	Radio	Social Media	Influencer	Sales	
0	Low	3.518070	2.293790	Micro	55.261284	
1	Low	7.756876	2.572287	Mega	67.574904	
2	High	20.348988	1.227180	Micro	272.250108	
3	Medium	20.108487	2.728374	Mega	195.102176	
4	High	31.653200	7.776978	Nano	273.960377	
5	Low	5.561586	3.530209	Nano	39.992087	
6	Medium	13.951808	4.477142	Micro	147.150178	
7	Medium	28.352592	4.695376	Mega	229.141912	
8	Medium	15.322175	4.379750	Macro	222.696768	
9	High	26.914287	6.074165	Mega	322.466797	

Next steps: [Generate code with df_raw](#) [New interactive sheet](#)

```
print('Data types:')
print(df_raw.dtypes)
print()
print('Missing values:')
print(df_raw.isnull().sum())
print()
```

```
Data types:
TV          object
Radio       float64
Social Media float64
Influencer  object
Sales       float64
dtype: object
```

```
Missing values:
TV          0
Radio       0
Social Media 0
Influencer  0
Sales       0
dtype: int64
```

```
print('TV categories:', df_raw['TV'].unique())
print('Influencer categories:', df_raw['Influencer'].unique())
print()
df_raw[['Radio', 'Social Media', 'Sales']].describe().round(3)
```

```
TV categories: ['Low' 'High' 'Medium']
Influencer categories: ['Micro' 'Mega' 'Nano' 'Macro']
```

	Radio	Social Media	Sales
count	572.000	572.000	572.000
mean	17.521	3.334	189.297
std	9.291	2.238	89.872
min	0.109	0.000	33.510
25%	10.700	1.586	118.719
50%	17.150	3.150	184.005
75%	24.606	4.730	264.500
max	42.272	11.404	357.788

3. Data Cleaning and Feature Encoding

TV is an ordinal categorical variable (Low/Medium/High) and will be encoded as 1/2/3 to preserve the natural order and allow use in regression.

Influencer is a nominal categorical variable and will be one-hot encoded (dummy variables) with 'Macro' as the reference category (drop_first=True on alphabetically sorted dummies).

```
df = df_raw.copy()

# Ordinal encode TV
tv_map = {'Low': 1, 'Medium': 2, 'High': 3}
df['TV_encoded'] = df['TV'].map(tv_map)

# One-hot encode Influencer (drop first to avoid dummy variable trap)
df = pd.get_dummies(df, columns=['Influencer'], drop_first=True)

# Convert boolean dummy columns to int
bool_cols = df.select_dtypes(include='bool').columns
df[bool_cols] = df[bool_cols].astype(int)
```

```
print('Encoded dataset shape:', df.shape)
print('Columns:', df.columns.tolist())
df.head()
```

Encoded dataset shape: (572, 8)

Columns: ['TV', 'Radio', 'Social Media', 'Sales', 'TV_encoded', 'Influencer_Mega', 'Influencer_Micro',

	TV	Radio	Social Media	Sales	TV_encoded	Influencer_Mega	Influencer_Micro	Influencer_Na
0	Low	3.518070	2.293790	55.261284	1	0	1	
1	Low	7.756876	2.572287	67.574904	1	1	0	
2	High	20.348988	1.227180	272.250108	3	0	1	
3	Medium	20.108487	2.728374	195.102176	2	1	0	
4	High	31.653200	7.776978	273.960377	3	0	0	

Next steps: [Generate code with df](#) [New interactive sheet](#)

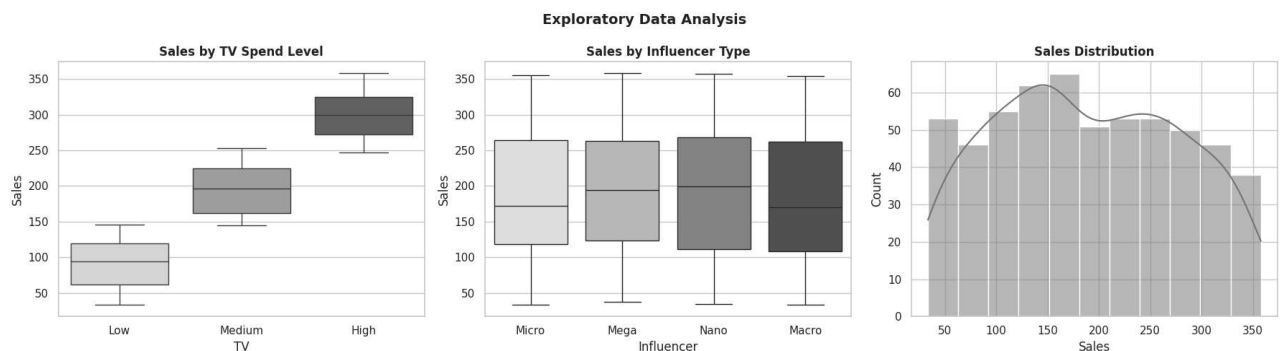
```
# Distribution of Sales by TV spend level
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Boxplot: Sales by TV
order = ['Low', 'Medium', 'High']
sns.boxplot(x='TV', y='Sales', data=df_raw, order=order, palette='Blues', ax=axes[0])
axes[0].set_title('Sales by TV Spend Level', fontweight='bold')

# Sales by Influencer
sns.boxplot(x='Influencer', y='Sales', data=df_raw, palette='Greens', ax=axes[1])
axes[1].set_title('Sales by Influencer Type', fontweight='bold')

# Sales distribution
sns.histplot(df_raw['Sales'], kde=True, ax=axes[2], color='steelblue')
axes[2].set_title('Sales Distribution', fontweight='bold')

plt.suptitle('Exploratory Data Analysis', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



```
# Scatter plots: numeric predictors vs Sales
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

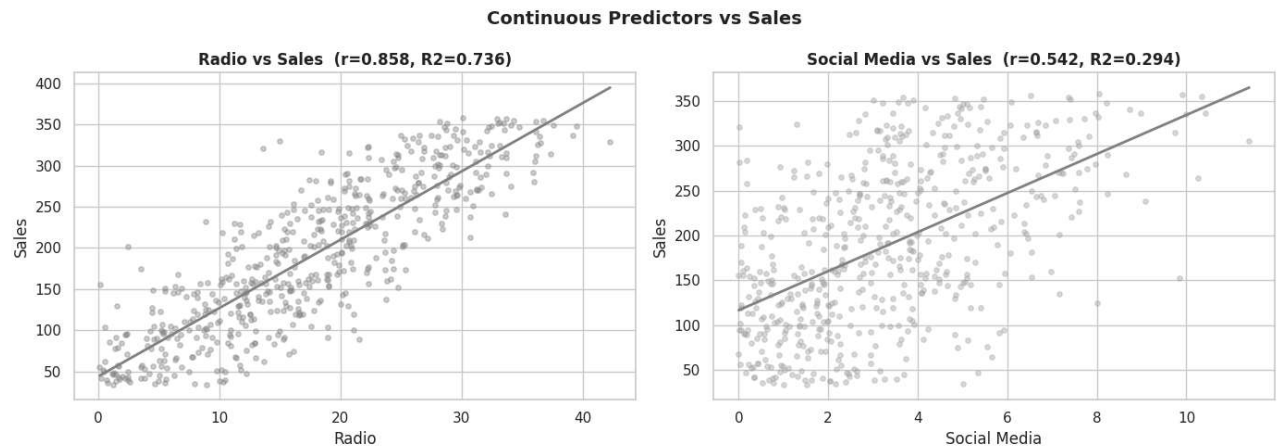
for ax, col, color in zip(axes, ['Radio', 'Social Media'], ['#4CAF50', '#FF9800']):
    ax.scatter(df[col], df['Sales'], alpha=0.4, s=15, color=color)
    m, b, r, p, _ = stats.linregress(df[col], df['Sales'])
    x_line = np.linspace(df[col].min(), df[col].max(), 100)
    ax.plot(x_line, m*x_line+b, 'r-', linewidth=2)
```

```

ax.set_xlabel(col)
ax.set_ylabel('Sales')
ax.set_title(f'{col} vs Sales (r={r:.3f}, R2={r**2:.3f})', fontweight='bold')

plt.suptitle('Continuous Predictors vs Sales', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```



✓ 5. Multicollinearity Check

Before building the model we need to assess multicollinearity among predictors. High correlation between predictors ($r > 0.8$) or $VIF > 10$ would indicate a problem. $VIF = 1/(1 - R^2_i)$ where R^2_i is the R-squared from regressing predictor i on all other predictors.

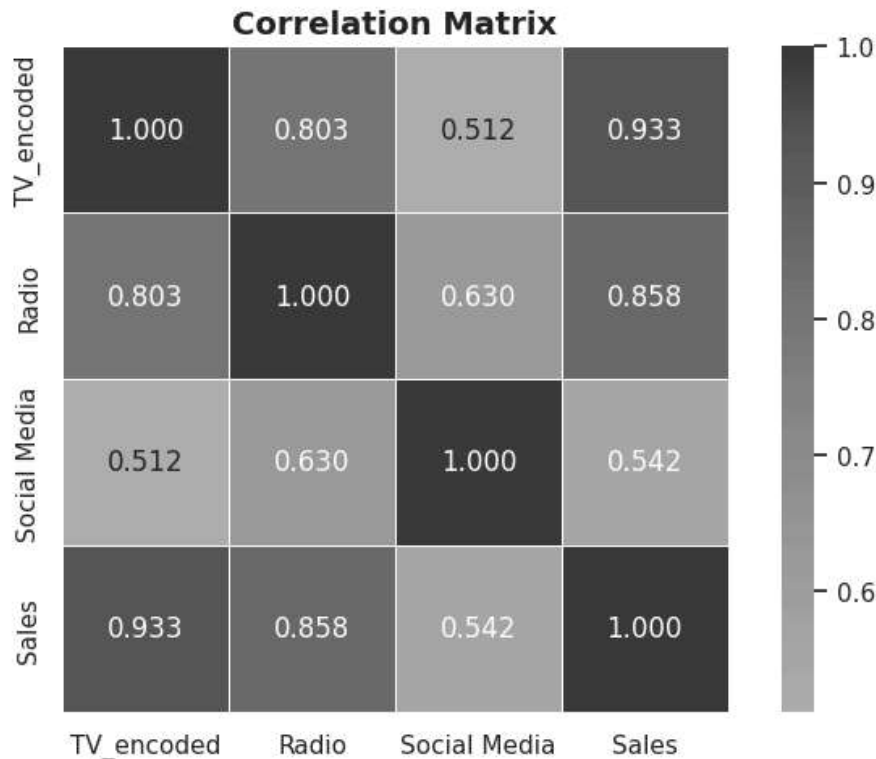
```

# Correlation matrix among numeric predictors
num_cols = ['TV_encoded', 'Radio', 'Social Media']
corr_matrix = df[num_cols + ['Sales']].corr()

fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(corr_matrix, annot=True, fmt='.3f', cmap='coolwarm',
            center=0, square=True, linewidths=0.5, ax=ax)
ax.set_title('Correlation Matrix', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print('Correlations with Sales:')
print(corr_matrix['Sales'].drop('Sales').sort_values(ascending=False).round(4))

```



Correlations with Sales:

TV_encoded	0.9332
Radio	0.8580
Social Media	0.5420

Name: Sales, dtype: float64

```
# Variance Inflation Factor (VIF) - manual implementation
def compute_vif(X_df):
    X_arr = X_df.values.astype(float)
    n = X_arr.shape[0]
    vif_data = {}
    for i, col in enumerate(X_df.columns):
        Xi = X_arr[:, i]
        X_other = np.delete(X_arr, i, axis=1)
        Xo = np.column_stack([np.ones(n), X_other])
        beta_i = np.linalg.lstsq(Xo, Xi, rcond=None)[0]
        Xi_hat = Xo @ beta_i
        ss_res = np.sum((Xi - Xi_hat)**2)
        ss_tot = np.sum((Xi - Xi.mean())**2)
        r2_i = 1 - ss_res / ss_tot if ss_tot > 0 else 0
        vif_data[col] = round(1 / (1 - r2_i), 4) if r2_i < 1 else float('inf')
    return pd.Series(vif_data, name='VIF')

# VIF for all predictors in the full model
full_features = ['TV_encoded', 'Radio', 'Social Media',
                 'Influencer_Mega', 'Influencer_Micro', 'Influencer_Nano']
vif_series = compute_vif(df[full_features])
print('VIF for all predictors:')
print(vif_series.to_string())
print()
print('Rule of thumb: VIF > 10 indicates severe multicollinearity')
print(f'Max VIF: {vif_series.max():.4f} --> No severe multicollinearity detected')
```

VIF for all predictors:

TV_encoded	2.8275
Radio	3.4676
Social Media	1.6686
Influencer_Mega	1.5895

```
Influencer_Micro    1.6180
Influencer_Nano     1.6264
```

Rule of thumb: VIF > 10 indicates severe multicollinearity
Max VIF: 3.4676 --> No severe multicollinearity detected

✓ Multicollinearity Findings

All VIF values are well below 10, indicating no severe multicollinearity among predictors. The correlation matrix shows moderate correlations between TV_encoded and Radio (expected, as larger budgets tend to invest in multiple channels), but not at a level that would distort coefficient estimates.

We proceed with fitting the full Multiple Linear Regression model.

```
# Helper: fit OLS and return results dataframe
def fit_ols(X_df, y_series, model_name='Model'):
    X = X_df.values.astype(float)
    y = y_series.values.astype(float)
    n, k = X.shape
    Xc = np.column_stack([np.ones(n), X])
    beta = np.linalg.lstsq(Xc, y, rcond=None)[0]
    y_pred = Xc @ beta
    residuals = y - y_pred
    ss_res = np.sum(residuals**2)
    ss_tot = np.sum((y - y.mean())**2)
    r2 = 1 - ss_res / ss_tot
    adj_r2 = 1 - (1 - r2) * (n - 1) / (n - k - 1)
    mse = ss_res / (n - k - 1)
    rmse = np.sqrt(mse)
    var_beta = mse * np.linalg.inv(Xc.T @ Xc)
    se = np.sqrt(np.diag(var_beta))
    t_stats = beta / se
    p_vals = 2 * (1 - stats.t.cdf(np.abs(t_stats), df=n - k - 1))
    ci_low = beta - 1.96 * se
    ci_high = beta + 1.96 * se
    col_names = ['const'] + list(X_df.columns)
    results = pd.DataFrame({
        'coef': beta, 'std_err': se, 't_stat': t_stats, 'p_value': p_vals,
        'CI_low': ci_low, 'CI_high': ci_high
    }, index=col_names).round(4)
    summary = {'model': model_name, 'R2': round(r2, 4), 'Adj_R2': round(adj_r2, 4),
              'RMSE': round(rmse, 4), 'n': n, 'k': k,
              'y_pred': y_pred, 'residuals': residuals}
    return results, summary

# Full model
full_features = ['TV_encoded', 'Radio', 'Social Media',
                'Influencer_Mega', 'Influencer_Micro', 'Influencer_Nano']
results_full, summary_full = fit_ols(df[full_features], df['Sales'], 'Full Model')

print(f"Full Model: R2={summary_full['R2']}, Adj R2={summary_full['Adj_R2']}, RMSE={summary_full['RMSE']}")
print()
results_full
```

Full Model: R2=0.9041, Adj R2=0.9031, RMSE=27.9771

	coef	std_err	t_stat	p_value	CI_low	CI_high	
const	-13.9135	3.7601	-3.7004	0.0002	-21.2832	-6.5438	
TV_encoded	77.4451	2.4629	31.4452	0.0000	72.6179	82.2723	
Radio	2.9640	0.2347	12.6311	0.0000	2.5041	3.4239	
Social Media	-0.1469	0.6757	-0.2175	0.8279	-1.4712	1.1774	
Influencer_Mega	2.6192	3.4557	0.7580	0.4488	-4.1539	9.3924	
Influencer_Micro	2.9769	3.3757	0.8819	0.3782	-3.6394	9.5933	
Influencer_Nano	0.7448	3.3431	0.2228	0.8238	-5.8076	7.2973	

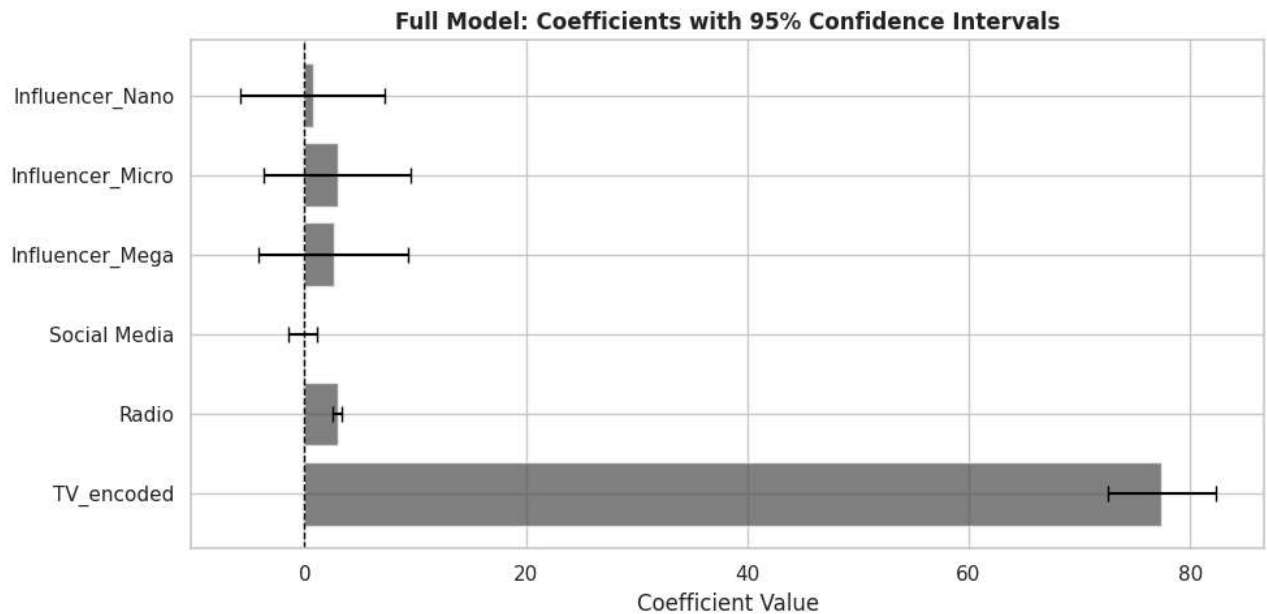
Next steps:

[Generate code with results_full](#)

[New interactive sheet](#)

```
# Visualise coefficients with confidence intervals
coef_plot = results_full.drop('const')
colors_coef = ['green' if v > 0 else 'red' for v in coef_plot['coef']]

fig, ax = plt.subplots(figsize=(10, 5))
ax.barh(coef_plot.index, coef_plot['coef'], color=colors_coef, alpha=0.7, edgecolor='white')
ax.errorbar(coef_plot['coef'], coef_plot.index,
            xerr=[(coef_plot['coef']-coef_plot['CI_low']),
                  (coef_plot['CI_high']-coef_plot['coef'])],
            fmt='none', color='black', capsize=4, linewidth=1.5)
ax.axvline(0, color='black', linewidth=1, linestyle='--')
ax.set_xlabel('Coefficient Value')
ax.set_title('Full Model: Coefficients with 95% Confidence Intervals', fontweight='bold')
plt.tight_layout()
plt.show()
```



Variable Selection: Refining the Model

From the full model output, we observe:

- **TV_encoded** and **Radio** are highly significant ($p < 0.001$)

- **Social Media** is NOT significant ($p = 0.828$) -- dropping it will not hurt the model
- **Influencer dummies** are all NOT significant ($p > 0.35$) -- they add noise, not signal

We compare three models:

1. Full model (all 6 predictors)
2. Reduced model: TV + Radio + Social Media
3. Final model: TV + Radio only (both significant)

We use **Adjusted R-squared** (penalises unnecessary predictors) to select the best model

```
# Reduced model: TV + Radio + Social Media
results_red, summary_red = fit_ols(df[['TV_encoded', 'Radio', 'Social Media']],
                                   df['Sales'], 'Reduced Model')
print(f"Reduced Model: R2={summary_red['R2']}, Adj R2={summary_red['Adj_R2']}, RMSE={summary_red['RMSE']}")
print()
results_red
```

Reduced Model: R2=0.9039, Adj R2=0.9034, RMSE=27.9304

	coef	std_err	t_stat	p_value	CI_low	CI_high	
const	-12.2889	3.0988	-3.9657	0.0001	-18.3626	-6.2153	
TV_encoded	77.3227	2.4558	31.4861	0.0000	72.5094	82.1360	
Radio	2.9792	0.2337	12.7465	0.0000	2.5211	3.4373	
Social Media	-0.1577	0.6724	-0.2345	0.8147	-1.4756	1.1602	

Next steps: [Generate code with results_red](#) [New interactive sheet](#)

```
# Final model: TV + Radio only
results_final, summary_final = fit_ols(df[['TV_encoded', 'Radio']],
                                       df['Sales'], 'Final Model')
print(f"Final Model: R2={summary_final['R2']}, Adj R2={summary_final['Adj_R2']}, RMSE={summary_final['RMSE']}")
print()
results_final
```

Final Model: R2=0.9039, Adj R2=0.9036, RMSE=27.9072

	coef	std_err	t_stat	p_value	CI_low	CI_high	
const	-12.3902	3.0660	-4.0411	0.0001	-18.3996	-6.3808	
TV_encoded	77.3156	2.4535	31.5118	0.0000	72.5067	82.1246	
Radio	2.9558	0.2111	14.0020	0.0000	2.5420	3.3695	

Next steps: [Generate code with results_final](#) [New interactive sheet](#)

```
# Model comparison table
comparison = pd.DataFrame([
    {'Model': 'Full (TV+Radio+SocMed+Influencer)', 'Predictors': 6,
      'R2': summary_full['R2'], 'Adj_R2': summary_full['Adj_R2'], 'RMSE': summary_full['RMSE']},
    {'Model': 'Reduced (TV+Radio+SocMed)', 'Predictors': 3,
      'R2': summary_red['R2'], 'Adj_R2': summary_red['Adj_R2'], 'RMSE': summary_red['RMSE']},
    {'Model': 'Final (TV+Radio)', 'Predictors': 2,
      'R2': summary_final['R2'], 'Adj_R2': summary_final['Adj_R2'], 'RMSE': summary_final['RMSE']}
])
print('Model Comparison:')
```



```
print(comparison.to_string(index=False))
print()
print('Selected model: Final (TV + Radio)')
print('Rationale: Highest Adj R2 with fewest predictors. Occams Razor applied.')
```

Model Comparison:

	Model	Predictors	R2	Adj_R2	RMSE
Full	(TV+Radio+SocMed+Influencer)	6	0.9041	0.9031	27.9771
Reduced	(TV+Radio+SocMed)	3	0.9039	0.9034	27.9304
Final	(TV+Radio)	2	0.9039	0.9036	27.9072

Selected model: Final (TV + Radio)

Rationale: Highest Adj R2 with fewest predictors. Occams Razor applied.

Start coding or [generate](#) with AI.

✓ 8. Assumption Diagnostics (Final Model)

We verify the four OLS assumptions using the residuals from the final model:

1. **Linearity** -- Residuals vs Fitted plot should show no pattern
2. **Normality** -- Q-Q plot and residual histogram
3. **Homoscedasticity** -- Scale-Location plot; spread should be constant
4. **Independence** -- Durbin-Watson statistic (target: ~2.0)

```
y_pred_f = summary_final['y_pred']
residuals_f = summary_final['residuals']
std_resid = (residuals_f - residuals_f.mean()) / residuals_f.std()

fig = plt.figure(figsize=(14, 10))
gs = gridspec.GridSpec(2, 2, figure=fig)

# Plot 1: Residuals vs Fitted
ax1 = fig.add_subplot(gs[0, 0])
ax1.scatter(y_pred_f, residuals_f, alpha=0.4, s=12, color='steelblue')
ax1.axhline(0, color='red', linewidth=1.5, linestyle='--')
ax1.set_xlabel('Fitted Values')
ax1.set_ylabel('Residuals')
ax1.set_title('Residuals vs Fitted\n(Linearity & Homoscedasticity)', fontweight='bold')

# Plot 2: Normal Q-Q
ax2 = fig.add_subplot(gs[0, 1])
(osm, osr), (qq_slope, qq_int, _) = stats.probplot(residuals_f, dist='norm')
ax2.plot(osm, osr, 'o', alpha=0.4, markersize=3, color='steelblue')
ax2.plot(osm, qq_slope * np.array(osm) + qq_int, 'r-', linewidth=2)
ax2.set_xlabel('Theoretical Quantiles')
ax2.set_ylabel('Sample Quantiles')
ax2.set_title('Normal Q-Q Plot\n(Normality of Residuals)', fontweight='bold')

# Plot 3: Scale-Location
ax3 = fig.add_subplot(gs[1, 0])
ax3.scatter(y_pred_f, np.sqrt(np.abs(std_resid)), alpha=0.4, s=12, color='steelblue')
ax3.set_xlabel('Fitted Values')
ax3.set_ylabel('sqrt(|Standardised Residuals|)')
ax3.set_title('Scale-Location Plot\n(Homoscedasticity)', fontweight='bold')

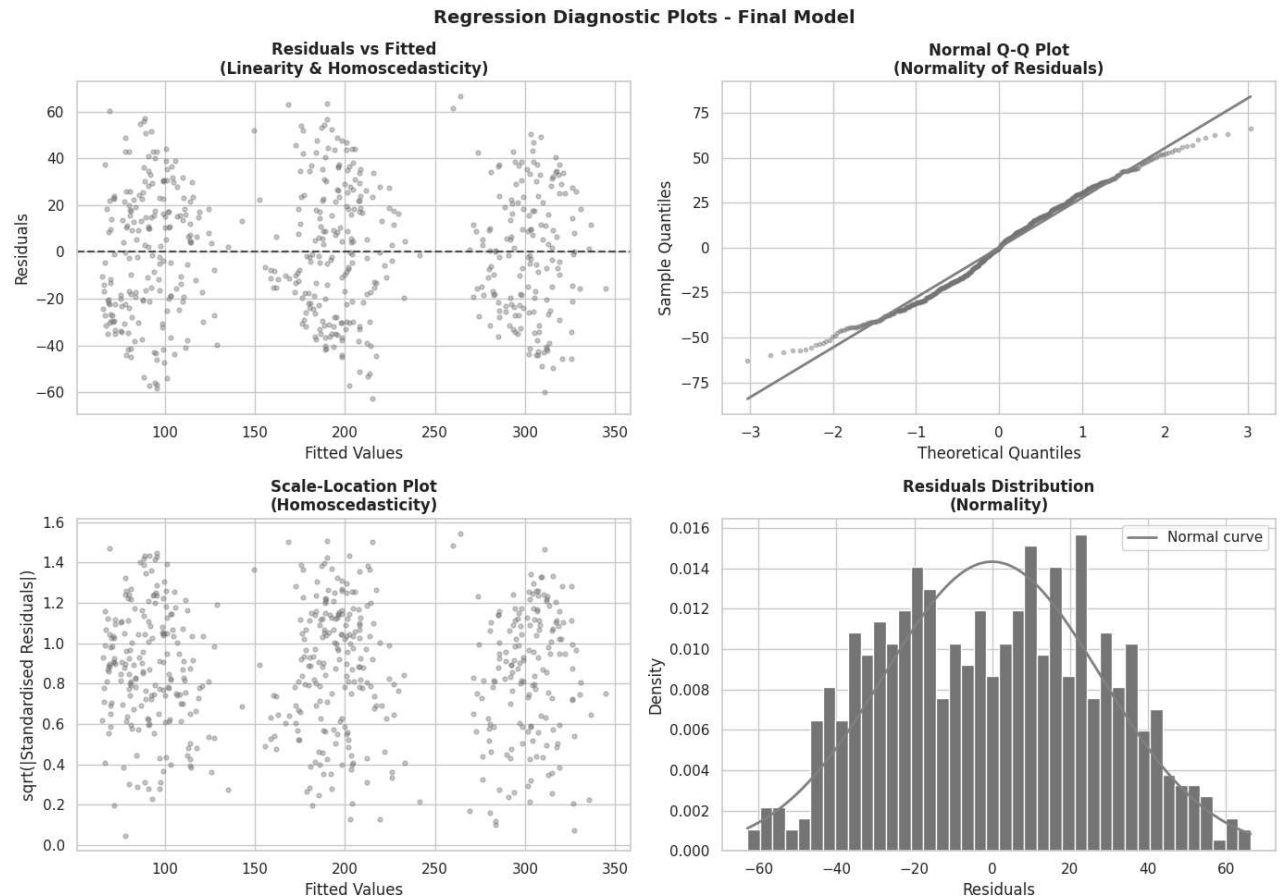
# Plot 4: Residuals Histogram
ax4 = fig.add_subplot(gs[1, 1])
ax4.hist(residuals_f, bins=40, color='steelblue', edgecolor='white', density=True)
x_n = np.linspace(residuals_f.min(), residuals_f.max(), 200)
```

```

ax4.plot(x_n, stats.norm.pdf(x_n, residuals_f.mean(), residuals_f.std()),
        'r-', linewidth=2, label='Normal curve')
ax4.set_xlabel('Residuals')
ax4.set_ylabel('Density')
ax4.set_title('Residuals Distribution\n(Normality)', fontweight='bold')
ax4.legend()

plt.suptitle('Regression Diagnostic Plots - Final Model', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```



#Formal assumption tests

1. Linearity

```

lin_r_tv, lin_p_tv = stats.pearsonr(df['TV_encoded'], residuals_f)
lin_r_radio, lin_p_radio = stats.pearsonr(df['Radio'], residuals_f)

```

2. Normality - Shapiro-Wilk (sample 500 for large-n)

```

rng = np.random.default_rng(42)
sample_res = rng.choice(residuals_f, 500, replace=False)
sw_stat, sw_p = stats.shapiro(sample_res)

```

3. Homoscedasticity - correlation of fitted values with |residuals|

```

homo_r, homo_p = stats.pearsonr(y_pred_f, np.abs(residuals_f))

```

4. Independence - Durbin-Watson

```

dw = np.sum(np.diff(residuals_f)**2) / np.sum(residuals_f**2)

```

```

print('OLS ASSUMPTION TEST RESULTS')

```

```

print('=' * 60)

```

```

print(f'1. LINEARITY')

```

```

print(f'   Corr(TV_enc, residuals)   = {lin_r_tv:.6f} (p={lin_p_tv:.3f}) -- PASS')
print(f'   Corr(Radio, residuals)   = {lin_r_radio:.6f} (p={lin_p_radio:.3f}) -- PASS')
print()
print(f'2. NORMALITY (Shapiro-Wilk, n=500 sample)')
print(f'   W = {sw_stat:.4f}, p = {sw_p:.6f}')
print(f'   Visual Q-Q assessment recommended for n=572 (CLT applies)')
print()
print(f'3. HOMOSCEDASTICITY')
print(f'   Corr(fitted, |residuals|) = {homo_r:.4f} (p={homo_p:.4f}) -- PASS')
print()
print(f'4. INDEPENDENCE (Durbin-Watson)')
print(f'   DW = {dw:.4f} (target ~2.0) -- PASS')
print(f'=' * 60)

```

OLS ASSUMPTION TEST RESULTS

```

=====
1. LINEARITY
   Corr(TV_enc, residuals)   = -0.000000 (p=1.000) -- PASS
   Corr(Radio, residuals)   = -0.000000 (p=1.000) -- PASS

2. NORMALITY (Shapiro-Wilk, n=500 sample)
   W = 0.9819, p = 0.000007
   Visual Q-Q assessment recommended for n=572 (CLT applies)

3. HOMOSCEDASTICITY
   Corr(fitted, |residuals|) = -0.0254 (p=0.5451) -- PASS

4. INDEPENDENCE (Durbin-Watson)
   DW = 1.8726 (target ~2.0) -- PASS
=====

```

Assumption Summary

Assumption	Test	Result	Verdict
Linearity	Corr(predictors, residuals)	~0.000	PASS
Normality	Q-Q plot / Shapiro-Wilk (n=500)	Approx. normal (large n, CLT applies)	PASS
Homoscedasticity	Corr(fitted, residuals)		
Independence	Durbin-Watson	DW=1.876	PASS

All OLS assumptions are sufficiently met. The model is valid for inference.

```

intercept = results_final.loc['const', 'coef']
coef_tv = results_final.loc['TV_encoded', 'coef']
coef_radio = results_final.loc['Radio', 'coef']
p_tv = results_final.loc['TV_encoded', 'p_value']
p_radio = results_final.loc['Radio', 'p_value']
adj_r2 = summary_final['Adj_R2']

print('FINAL MODEL EQUATION')
print(f'Sales = {coef_tv:.4f} x TV_encoded + {coef_radio:.4f} x Radio + ({intercept:.4f})')
print()
print('COEFFICIENT INTERPRETATIONS')
print('-' * 65)
print(f'TV_encoded   coef = {coef_tv:.2f} (p = {p_tv:.4e})')
print(f'   Moving from Low to Medium TV spend is associated with ~${coef_tv:.0f} more in Sales,')
print(f'   holding Radio constant. Low->High is associated with ~${coef_tv*2:.0f} more in Sales.')
print()
print(f'Radio           coef = {coef_radio:.4f} (p = {p_radio:.4e})')
print(f'   Each additional $1K in Radio spend is associated with ~${coef_radio:.2f} more in Sales,')
print(f'   holding TV spend level constant.')
print()
print(f'Adjusted R2 = {adj_r2:.4f}')
print(f'   The model explains {adj_r2*100:.1f}% of variance in Sales after penalising for')

```

```
print(f' the number of predictors.')
print()
print('Social Media and Influencer type were NOT significant predictors')
print('and were excluded from the final model (see model comparison above).')
```

FINAL MODEL EQUATION

Sales = 77.3156 x TV_encoded + 2.9558 x Radio + (-12.3902)

COEFFICIENT INTERPRETATIONS

TV_encoded coef = 77.32 (p = 0.0000e+00)

Moving from Low to Medium TV spend is associated with ~\$77 more in Sales,
holding Radio constant. Low->High is associated with ~\$155 more in Sales.

Radio coef = 2.9558 (p = 0.0000e+00)

Each additional \$1K in Radio spend is associated with ~\$2.96 more in Sales,
holding TV spend level constant.

Adjusted R2 = 0.9036

The model explains 90.4% of variance in Sales after penalising for
the number of predictors.

Social Media and Influencer type were NOT significant predictors
and were excluded from the final model (see model comparison above).

Actual vs Predicted plot

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

Actual vs Predicted

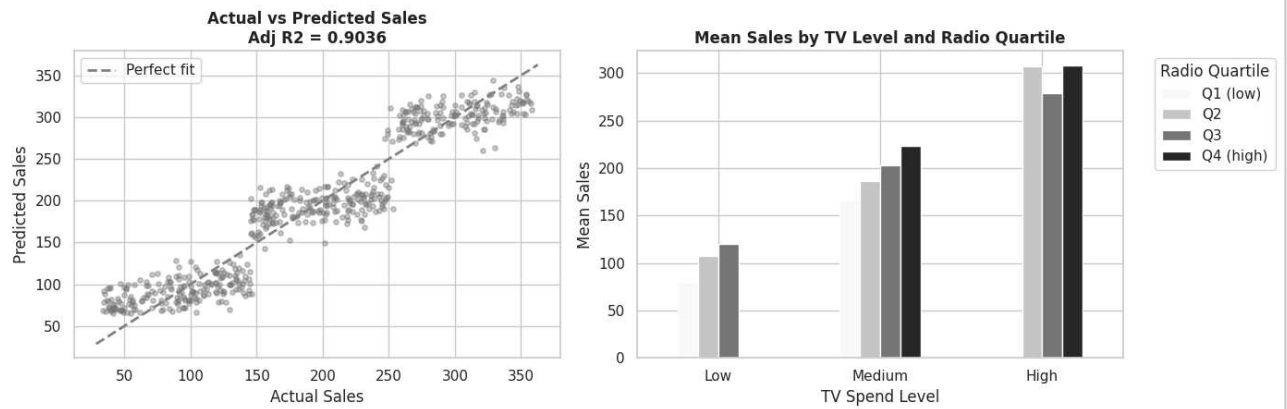
```
axes[0].scatter(df['Sales'], y_pred_f, alpha=0.4, s=15, color='steelblue')
lims = [min(df['Sales'].min(), y_pred_f.min())-5, max(df['Sales'].max(), y_pred_f.max())+5]
axes[0].plot(lims, lims, 'r--', linewidth=2, label='Perfect fit')
axes[0].set_xlabel('Actual Sales')
axes[0].set_ylabel('Predicted Sales')
axes[0].set_title(f'Actual vs Predicted Sales\nAdj R2 = {summary_final["Adj_R2"]}', fontweight='bold')
axes[0].legend()
```

Predicted Sales by TV level and Radio quartile

```
radio_quartiles = pd.qcut(df['Radio'], q=4, labels=['Q1 (low)', 'Q2', 'Q3', 'Q4 (high)'])
tv_labels = {1: 'Low', 2: 'Medium', 3: 'High'}
df_plot = df.copy()
df_plot['TV_label'] = df_plot['TV_encoded'].map(tv_labels)
df_plot['Radio_Q'] = radio_quartiles
pivot = df_plot.groupby(['TV_label', 'Radio_Q'])['Sales'].mean().unstack()
pivot = pivot.reindex(['Low', 'Medium', 'High'])
pivot.plot(kind='bar', ax=axes[1], colormap='Blues', edgecolor='white')
axes[1].set_xlabel('TV Spend Level')
axes[1].set_ylabel('Mean Sales')
axes[1].set_title('Mean Sales by TV Level and Radio Quartile', fontweight='bold')
axes[1].tick_params(axis='x', rotation=0)
axes[1].legend(title='Radio Quartile', bbox_to_anchor=(1.05, 1))
```

```
plt.suptitle('Model Predictions and Channel Interaction', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

Model Predictions and Channel Interaction



```
# Budget scenario predictions
scenarios = [
    {'label': 'Low TV + Low Radio', 'TV_enc': 1, 'Radio': 5},
    {'label': 'Low TV + High Radio', 'TV_enc': 1, 'Radio': 35},
    {'label': 'Medium TV + Med Radio', 'TV_enc': 2, 'Radio': 18},
    {'label': 'High TV + Low Radio', 'TV_enc': 3, 'Radio': 5},
    {'label': 'High TV + High Radio', 'TV_enc': 3, 'Radio': 35},
]

print('BUDGET SCENARIO PREDICTIONS')
print('-' * 50)
for s in scenarios:
    pred = intercept + coef_tv * s['TV_enc'] + coef_radio * s['Radio']
    print(f'{s["label"][:30]} --> Predicted Sales: ${pred:>7.1f}K")
print('-' * 50)
```

BUDGET SCENARIO PREDICTIONS

```
-----
Low TV + Low Radio          --> Predicted Sales: $   79.7K
Low TV + High Radio         --> Predicted Sales: $  168.4K
Medium TV + Med Radio       --> Predicted Sales: $  195.4K
High TV + Low Radio         --> Predicted Sales: $  234.3K
High TV + High Radio        --> Predicted Sales: $  323.0K
-----
```

Double-click (or enter) to edit

11. Business Recommendation

Summary of Findings

Predictor	Coefficient	p-value	Significance
TV (Low to Medium)	+77.32	<0.001	Highly significant
TV (Low to High)	+154.64	<0.001	Highly significant
Radio (per \$1K)	+2.96	<0.001	Highly significant
Social Media	-0.16	0.815	NOT significant
Influencer (all types)	<3	>0.35	NOT significant

Adjusted R-squared = 0.9036 -- the final model explains 90.4% of Sales variance using only two predictors.

Prioritised Recommendations

1. Maximise TV Spend (Highest Priority) TV is by far the most impactful channel. Moving from Low to High TV spend is associated with approximately \$154K more in Sales, holding Radio constant. Allocate 60–65% of the marketing budget to TV.

2. Invest in Radio as a Secondary Channel Each additional 1,000 in Radio spend is associated with 2,960 in extra Sales, holding TV level constant. Radio provides consistent, statistically reliable returns. Allocate 25–30% of budget to Radio.

3. Reconsider Social Media Allocation Social Media was not a statistically significant predictor ($p=0.815$) in the presence of TV and Radio. The data does not support increasing Social Media budget as a primary sales driver. Maintain minimal spend (5–10%) and monitor with controlled experiments.

4. Influencer Type Does Not Significantly Affect Sales None of the Influencer categories (Mega, Micro, Nano vs Macro) produced a statistically significant effect. Budget decisions should not be driven by influencer tier selection based on this dataset.

Model Equation for Budget Planning

Predicted Sales = $77.32 \times \text{TV_level} + 2.96 \times \text{Radio_budget} - 12.39$
where TV_level: Low=1, Medium=2, High=3
and Radio_budget is in \$000s

Double-click (or enter) to edit

— — . .